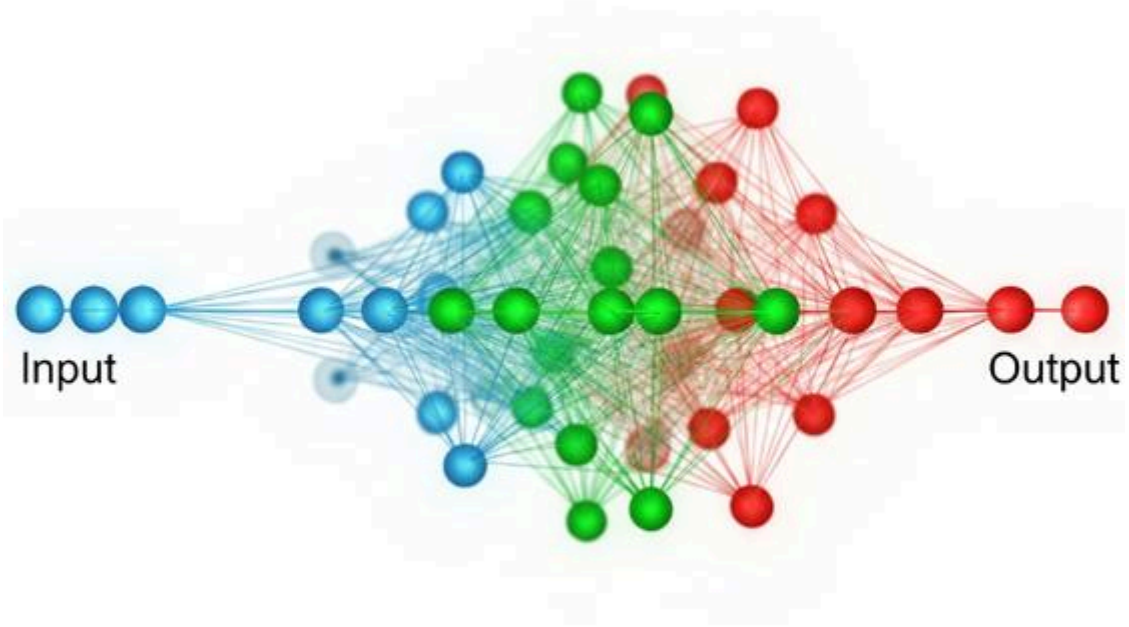
MEMORIA TÉCNICA PERCEPTRÓN MULTICAPA



Adrián García García **Modelo de Inteligencia Artificial**

ÍNDICE

1. DATASET
2. METODOLOGÍA
3. RESULTADOS
4. TIEMPOS DE CONVERGENCIA
5. RECONOCIMIENTO EN TIEMPO REAL (SIMULADO)
6. CONCLUSIÓN

DATASET

En este caso, he elegido el dataset EMNIST, principalmente porque veo más práctico e interesante entrenar un perceptrón multicapa que sea capaz de distinguir entre dígitos y letras. Hay que tener en cuenta que hay diferentes tipos de datasets EMNIST, como el byclass, bymerge, letters… En este caso, he elegido bymerge. La diferencia principal entre byclass y bymerge, es que hay muchas letras en mayúsculas y en minúsculas que se parecen, como la “C” y la “c”, o la “X” y la “x”, pues ese tipo de letras cuentan como la misma clase, y es precisamente lo que hace bymerge. Mientras que byclass las toma como diferente clase, es decir, la “C” y la “c” son clases distintas. Considero que bymerge es algo mejor, ya que diferenciar entre “C” y “c” no es tan relevante, sino lo interesante es que sepa que esa letra es la ce

En este dataset, viene un total de 814255 imágenes, de las cuales 697932 son de entrenamiento y 116323 son de test. Para este dataset hay un total de 47 clases, entre todos los dígitos y letras. Además, estas imágenes tienen un formato de 28x28 píxeles y están invertidas horizontalmente, por lo que luego en el código habrá que hacer transposición para que sea legible al humano

METODOLOGÍA

Para esta actividad, he hecho uso de librerías, ya que me parecía interesante aprender, en este caso, TensorFlow y Keras, que son librerías bastante potentes para entrenar redes neuronales, lo que supone una oportunidad para adentrarme en el mundo tanto de TensorFlow como de Keras

- ¿Qué es TensorFlow?

  Es una librería de código abierto de Google para machine learning y redes neuronales profundas, facilitando la creación e implementación de modelos de aprendizaje automático en diferentes plataformas como computadoras de escritorio, dispositivos móviles, la web y la nube

- ¿Qué es Keras?

  Es otra librería de código abierto escrita en Python, que nos permite también diseñar y acelerar la creación de modelos de redes neuronales. Keras es totalmente compatible con TensorFlow, lo que nos permite hacer redes neuronales con cierta complejidad

- ¿Qué clases tiene Keras que se utilicen en el código?

  - Dense: Representa la capa de neurona, donde cada neurona de dicha capa, está conectada con las neuronas de la capa anterior. Es

importante está clase, ya que nos permite crear las capas ocultas y la capa final

- Input: Con esta clase definimos la capa de entrada de la red neuronal, donde se define la forma y el tipo de datos que el modelo debe esperar (como por ejemplo, define qué tamaño de la imagen debe esperar)
- Model: Esta clase nos permite crear el modelo en sí, donde se le indica la capa de entrada y la capa de salida, encargándose de conectar todas las capas, creando finalmente la red neuronal
- Accuracy: Nos permite medir cuántas predicciones ha acertado con respecto a los datos de test

Ahora hablaremos de los distintos optimizadores que tiene Keras y que utilizamos en nuestro código. Recordemos que un optimizador es el responsable de actualizar los pesos del modelo durante el entrenamiento

- Adam: Es un optimizador que combina las mejores características de dos optimizadores, Momentum y RMSprop. Se utiliza en el aprendizaje profundo debido a su eficiencia y capacidades de tasa de aprendizaje adaptativo
- SGD: Este optimizador, también conocido como descenso de gradiente estocástico, obtiene un pequeño subconjunto de datos (mini-lote) para cada iteración y actualizar los parámetros, de esta manera ocupa menos memoria
- RMSprop: Es un algoritmo de optimización de la tasa de aprendizaje adaptativo diseñado para mejorar el rendimiento y la velocidad del entrenamiento de modelos de aprendizaje profundo. RMSProp fue desarrollado para abordar las limitaciones de métodos de optimización anteriores, como SGD ya que este último utiliza una tasa de aprendizaje constante que puede ser ineficiente

➢ EarlyStopping: Otra clase usada muy útil de Keras es EarlyStopping, esto nos permite dejar de entrenar (ya no hace otra vuelta de entrenamiento) si no mejora el rendimiento en la vuelta actual (se puede establecer qué métrica se quiere observar, como la pérdida, accuracy…). De esta manera podemos ahorrar tiempo al entrenar el modelo

➢ Compile: Este método de Keras permite configurar el modelo, añadiendo el optimizador, la función de pérdida y la métrica para evaluar el modelo

➢ Fit: Este método nos permite entrenar el modelo una vez configurado

➢ load_model: Este método nos facilita importar un modelo entrenado que tengamos guardado en nuestro ordenador

- ¿Qué métodos usamos de TensorFlow?
  - convert_image_dtype: Nos ayuda a escalar los píxeles de la imagen en un rango entre el 0 y el 1 para que la red neuronal pueda entrenar mejor
  - transpose: Nos posibilita transponer las imágenes, en este caso, como las imágenes vienen dadas invertidas horizontalmente, la transponemos para poder visualizarlo mejor posteriormente
  - reshape: Sirve para redimensionar, lo usaremos para redimensionar la imagen a un vector numérico, ya que los modelos reciben entradas vectoriales. Como las imágenes del dataset son de 28 x 28 píxeles, se redimensionan a vectores de 784 elementos
  - one_hot: Permite convertir la etiqueta entera (el label que se le pasa como parámetro) en un vector one-hot compatible con la salida softmax del modelo
  - load: Nos permite leer el dataset en cuestión, pudiendo dividirlo en datos de entrenamiento y de test

Por otro lado, en cuanto a la búsqueda de parámetros, me he decantado por hacer un GridSearch manual mediante bucles, ya que, investigando si era posible utilizar GridSearchCV con Keras y TensorFlow, he visto una clase llamada KerasClassifier que se comportaba como un “envoltorio” del modelo y se podía hacer uso de GridSearchCV, el problema es que KerasClassifier está obsoleto, y dependiendo de la versión que se tenga instalada de TensorFlow, puede funcionar o no. Sin embargo fue reemplazado por una librería llamada Sci-Keras, pero está en versiones muy tempranas (Pre-Alpha) por lo que he decidido hacer un GridSearch manual mediante bucles, para ello, he guardado en distintos arrays, los diferentes parámetros, que en este caso son:

- [ ] Número de capas y de neuronas: En este caso voy a comprobar estos número de capas y neuronas: [(128,), (256,), (256, 128), (256, 256)], he elegido este rango ya que primero comprobamos si el modelo es capaz de resolver el problema con mayor puntuación con una sola capa, luego se comprueba si mejora el rendimiento con una capa oculta más. En cuanto al número de las neuronas, es un número aceptable para imágenes de tamaño 28 x 28
- [ ] Función de activación: Por otra parte, probaremos estas funciones de activación: ['relu', 'tanh', 'sigmoid']. Considero que son las más importantes, y podemos comprobar cuál funciona mejor para el reconocimiento de carácteres

- ☐ Tasa de aprendizaje: Para este otro parámetro, hacemos uso de las tasas de aprendizaje más comunes: [0.001, 0.01]. Primero se comprueba con un valor más estable (0.001), y luego se compara si con una tasa algo más agresiva aumenta o no el rendimiento del modelo
- ☐ Optimizadores: Por último, los optimizadores que se someten a pruebas son: [Adam, SGD, RMSprop]. He seleccionado estos ya que son optimizadores que se comportan de manera diferente (explicado anteriormente) y nos permite ver cuál se comporta mejor para este tipo de problemas

RESULTADOS

A continuación se harán diferentes tablas por número de capas y de neuronas mostrando los diferentes resultados

EL significado de las columnas son:

- ➔ Capas: El número de capas ocultas que tiene la red
- ➔ Unidades: El número de neuronas que tendrá cada capa oculta
- ➔ Activación: La función de activación de las capas ocultas
- ➔ Optimizador: Los distintos optimizadores que se pondrán a prueba
- ➔ LR: La tasa aprendizaje del modelo
- ➔ Val Acc: Es el accuracy que se calcula con los datos de validación
- ➔ Val Prec: Es el valor de la precisión con respecto a los datos de validación
- ➔ Val Loss: Es el error de validación
- ➔ Épocas: Es el número de vuelta que da por cada entrenamiento de un modelo
- ➔ Tiempo: El tiempo que tarda en terminar el entrenamiento

Esta tabla se corresponde con los resultados de los modelos que tiene una capa y 128 neuronas:

Capas	Unidades	Activación	Optimizador	LR	Val Acc	Val Prec	Val Loss	Épocas	Tiempo (s)
1	(128)	relu	Adam	0.001	0.8547	0.8817	0.4406	3	49.09
1	(128)	relu	SGD	0.001	0.4700	0.8971	2.2887	2	44.35
1	(128)	relu	RMSprop	0.001	0.8454	0.8775	0.4795	2	38.30
1	(128)	relu	Adam	0.01	0.8203	0.8456	0.5672	2	46.45

1	(128)	relu	SGD	0.01	0.6960	0.8376	1.0887	2	38.36
1	(128)	relu	RMSprop	0.01	0.8008	0.8279	0.6942	2	56.35
1	(128)	tanh	Adam	0.001	0.8452	0.8843	0.4777	2	45.17
1	(128)	tanh	SGD	0.001	0.4795	0.9284	2.2235	2	41.85
1	(128)	tanh	RMSprop	0.001	0.8426	0.8836	0.4825	2	44.36
1	(128)	tanh	Adam	0.01	0.8218	0.8534	0.5511	2	45.97
1	(128)	tanh	SGD	0.01	0.6783	0.8460	1.1692	2	41.01
1	(128)	tanh	RMSprop	0.01	0.8066	0.8309	0.6314	2	46.49
1	(128)	sigmoid	Adam	0.001	0.8261	0.8812	0.5618	2	41.74
1	(128)	sigmoid	SGD	0.001	0.3134	0.0000	3.3244	2	43.24
1	(128)	sigmoid	RMSprop	0.001	0.8278	0.8832	0.5574	2	42.37
1	(128)	sigmoid	Adam	0.01	0.8407	0.8663	0.4799	2	43.13
1	(128)	sigmoid	SGD	0.01	0.5198	0.8845	1.9570	2	46.36
1	(128)	sigmoid	RMSprop	0.01	0.8391	0.8630	0.4903	2	57.44

De estos resultados observamos que el optimizador SGD obtiene unos resultados muy bajos con respecto a Adam y RMSprop. Aunque el que mejores resultados obtiene es el optimizador Adam. Además, podemos observar que el tiempo de convergencia, es decir, lo que tarda el modelo en cuanto al número de iteraciones

(épocas), en mejorar y mantenerse estable, en todos los casos son dos iteraciones excepto el primer caso del optimizador Adam, que necesita tres iteraciones para mantenerse estable

Esta tabla se corresponde a los resultados de una capa y 256 neuronas

Capas	Unidades	Activación	Optimizador	LR	Val Acc	Val Prec	Val Loss	Épocas	Tiempo (s)
1	(256)	relu	Adam	0.001	0.8673	0.8890	0.3914	3	84.28
1	(256)	relu	SGD	0.001	0.4878	0.8843	2.1719	2	59.90
1	(256)	relu	RMSprop	0.001	0.8599	0.8817	0.4215	2	50.85
1	(256)	relu	Adam	0.01	0.8277	0.8520	0.5503	2	54.52
1	(256)	relu	SGD	0.01	0.7055	0.8408	1.0557	2	52.37
1	(256)	relu	RMSprop	0.01	0.8153	0.8392	0.6606	2	54.35
1	(256)	tanh	Adam	0.001	0.8604	0.8892	0.4216	2	47.35
1	(256)	tanh	SGD	0.001	0.5260	0.8855	2.0241	2	49.93
1	(256)	tanh	RMSprop	0.001	0.8543	0.8818	0.4378	2	52.91
1	(256)	tanh	Adam	0.01	0.8169	0.8385	0.6052	2	54.56
1	(256)	tanh	SGD	0.01	0.6932	0.8411	1.1077	2	50.27
1	(256)	tanh	RMSprop	0.01	0.7886	0.8114	0.7166	2	52.30
1	(256)	sigmoid	Adam	0.001	0.8467	0.8898	0.4775	2	54.92

1	(256)	sigmoid	SGD	0.001	0.3224	0.0000	3.2733	2	51.59
1	(256)	sigmoid	RMSprop	0.001	0.8421	0.8821	0.4932	2	69.00
1	(256)	sigmoid	Adam	0.01	0.8532	0.8762	0.4369	2	42.43
1	(256)	sigmoid	SGD	0.01	0.5514	0.8616	1.8073	2	54.02
1	(256)	sigmoid	RMSprop	0.01	0.8502	0.8685	0.4596	2	54.70

Estos resultados son mejores que los anteriores, pero sigue el mismo patrón, el optimizador SGD obtiene unos resultados muy bajos con respecto a Adam y RMSprop y el optimizador Adam es el que sigue obteniendo los mejores resultados. Algo curioso que podemos concluir viendo ambas tablas, es que el optimizador SGD es el único que funciona mejor cuando la tasa de aprendizaje es 0.01.

Con respecto al tiempo de convergencia, no ha cambiado con respecto a lo anterior, solo el primer caso de Adam es el que necesita tres iteraciones para mantener una estabilidad con buenos resultados, aunque en ambos casos es el modelo que mejore métrica obtiene en cuanto al accuracy

Ahora vamos a pasar directamente con la tabla de dos capas con 256 neuronas cada una, ya que las conclusiones son iguales con la tabla de dos capas con 256 neuronas en una capa y 126 neuronas en otra (adjunto link de la tabla), solo que en este caso que vamos a comentar, se obtiene mejores resultados

Capas	Unidades	Activación	Optimizador	LR	Val Acc	Val Prec	Val Loss	Épocas	Tiempo (s)
2	(256,256)	relu	Adam	0.001	0.8777	0.8939	0.3501	3	95.20
2	(256,256)	relu	SGD	0.001	0.4815	0.8625	2.1301	2	55.36
2	(256,256)	relu	RMSprop	0.001	0.8671	0.8886	0.3810	2	60.96

2	(256,256)	relu	Adam	0.01	0.8315	0.8663	0.5414	2	61.52
2	(256,256)	relu	SGD	0.01	0.7260	0.8362	0.9387	2	56.24
2	(256,256)	relu	RMSprop	0.01	0.7802	0.8170	0.7915	2	62.69
2	(256,256)	tanh	Adam	0.001	0.8666	0.8897	0.3845	2	62.00
2	(256,256)	tanh	SGD	0.001	0.5297	0.8802	1.9158	2	62.62
2	(256,256)	tanh	RMSprop	0.001	0.8634	0.8833	0.3900	2	62.31
2	(256,256)	tanh	Adam	0.01	0.7819	0.8170	0.6901	2	63.52
2	(256,256)	tanh	SGD	0.01	0.7162	0.8447	1.0094	2	61.91
2	(256,256)	tanh	RMSprop	0.01	0.7372	0.7587	0.9096	2	64.89
2	(256,256)	sigmoid	Adam	0.001	0.8595	0.8867	0.4182	2	60.32
2	(256,256)	sigmoid	SGD	0.001	0.0814	0.0000	3.5360	2	61.33
2	(256,256)	sigmoid	RMSprop	0.001	0.8441	0.8829	0.4678	2	60.49
2	(256,256)	sigmoid	Adam	0.01	0.8572	0.8810	0.4149	2	62.25
2	(256,256)	sigmoid	SGD	0.01	0.3614	0.0000	2.8469	2	61.95
2	(256,256)	sigmoid	RMSprop	0.01	0.8615	0.8800	0.4043	2	58.53

Podemos ver que en este caso con dos capas y 256 neuronas en ambas, Adam sigue siendo el mejor optimizador en cuanto a métrica de accuracy y en este caso, también de precisión. Sigue necesitando 3 épocas (sólo en el primer caso, donde se obtiene el mejor resultado) para obtener un buen resultado y estabilizarse con respecto a los demás optimizadores

A continuación se muestra una tabla resumen, con las mejores configuraciones por número de capas, número de neuronas y por tipo de función de activación

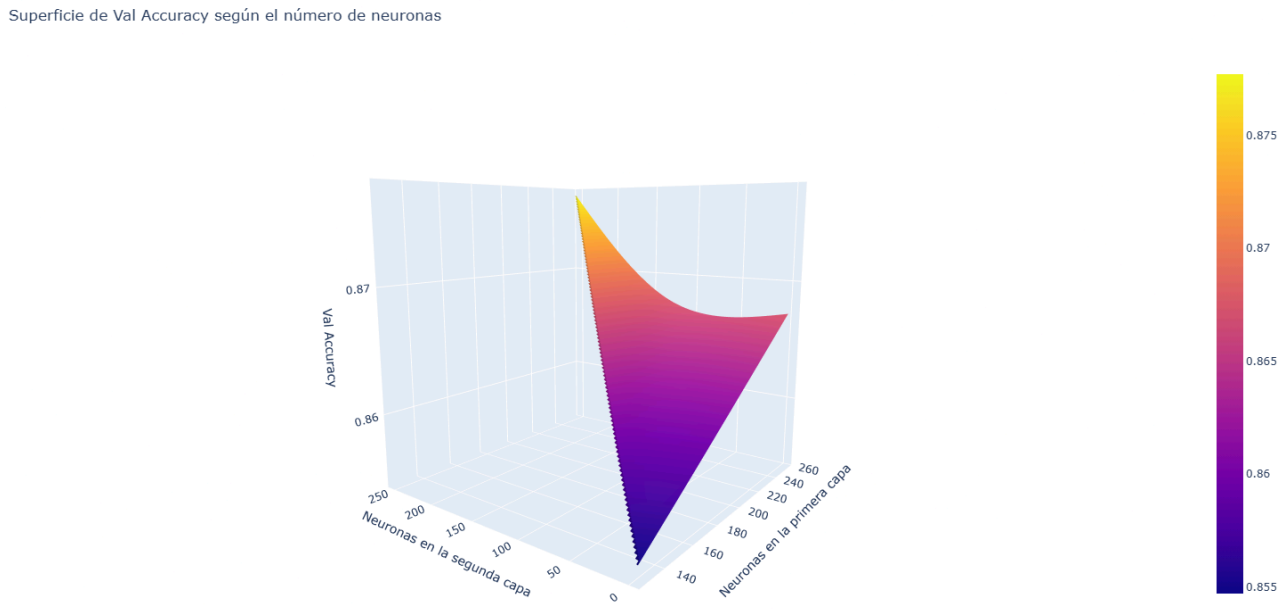
Capas	Unidades	Activación	Optimizador	LR	Val Accuracy	Val Precision	Val Loss	Épocas	Tiempo (s)
1	(128)	ReLU	Adam	0.001	0.8547	0.8817	0.4406	3	49.1
1	(128)	Tanh	Adam	0.001	0.8452	0.8843	0.4777	2	45.2
1	(128)	Sigmoid	Adam	0.01	0.8407	0.8663	0.4799	2	43.1
1	(256)	ReLU	Adam	0.001	0.8673	0.8890	0.3914	3	84.3
1	(256)	Tanh	Adam	0.001	0.8604	0.8892	0.4216	2	47.3
1	(256)	Sigmoid	Adam	0.01	0.8532	0.8762	0.4369	2	42.4
2	(256,128)	ReLU	RMSprop	0.001	0.8676	0.8872	0.3886	2	58.6
2	(256,128)	Tanh	Adam	0.001	0.8616	0.8873	0.4026	2	57.4
2	(256,128)	Sigmoid	RMSprop	0.01	0.8550	0.8730	0.4291	2	63.4

2	**(256,256)**	**ReLU**	**Adam**	**0.001**	**0.8777**	**0.8939**	**0.3501**	**3**	**95.2**
2	(256,256)	Tanh	Adam	0.001	0.8666	0.8897	0.3845	2	62.0
2	(256,256)	Sigmoid	RMSprop	0.01	0.8615	0.8800	0.4043	2	58.5

Vemos que el optimizador Adam es el predominante y el que mejor resultado obtiene en la mayoría, aunque el optimizador RMSprop también aparece varias veces con buenos resultados y tiempos de entrenamiento bastante adecuados

Otra conclusión que podemos extraer es que, al analizar las métricas de cada modelo, no se observa una diferencia significativa entre una capa de 256 neuronas, y dos capas de 256 neuronas, por lo que realmente nos podemos quedar con cualquiera, pero para ser justos, vamos a quedarnos con la que ha obtenido un mejor accuracy y una mejor precisión, que es el modelo de **dos capas con 256 neuronas** en cada una, con el optimizador **Adam**, una **tasa de 0.001** y la **función de activación ReLU**

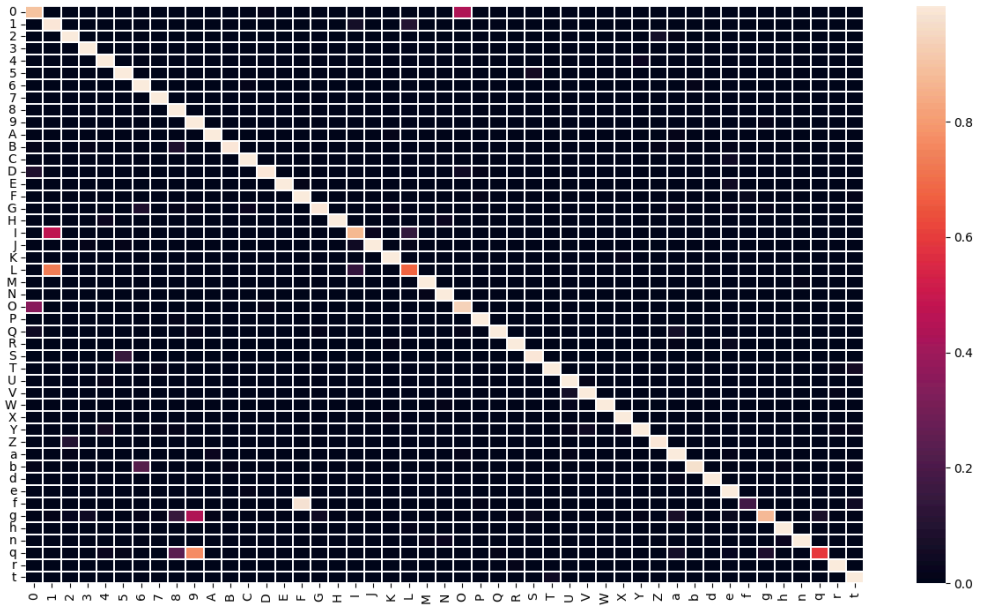
En esta ocasión, vemos un Surface Plot, donde se representa en el eje X las neuronas de la primera capa, en el eje Y se puede observar las neuronas de la segunda capa, y en el eje Z, el accuracy. Podemos ver que cuantas más neuronas en ambas capas, mejor accuracy obtenemos. Aunque podemos percibir una pequeña excepción cuando la segunda capa tiene 0 neuronas (es decir, solo hay una capa) y la primera capa tiene 256 neuronas, en este caso hay un pico máximo local donde tiene mejor accuracy que un modelo que tiene dos capas de 256 y 128 neuronas respectivamente. Pero como hemos visto también en la tabla anterior, el que obtiene un mejor accuracy es el de dos capas con 256 neuronas en cada una



Entrenamos el modelo con las características mencionadas anteriormente, y vemos que obtiene un accuracy de un 88%

```
La precici⍰n final del mejor modelo es de 0.8803160190582275
```

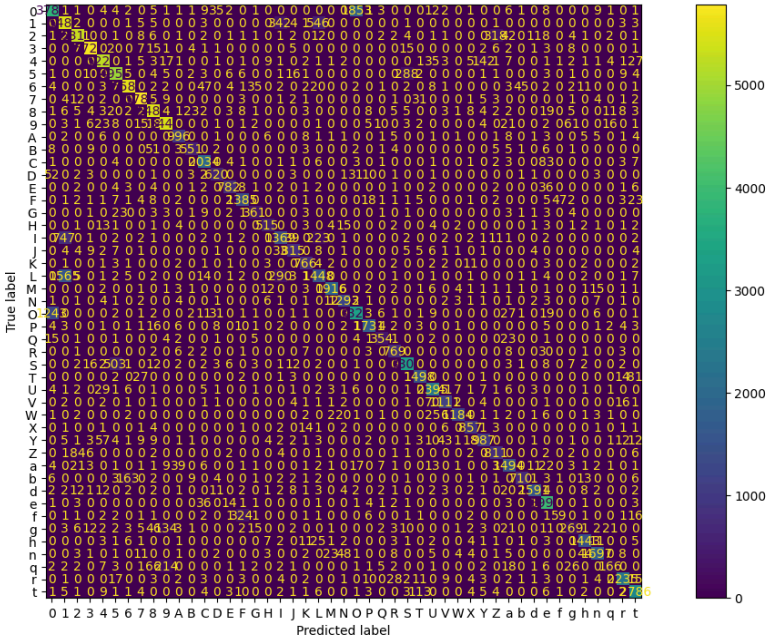
A continuación, vamos a visualizar la matriz de correlación, primero vamos a verlo de forma legible (la matriz normalizada entre 0 y 1)



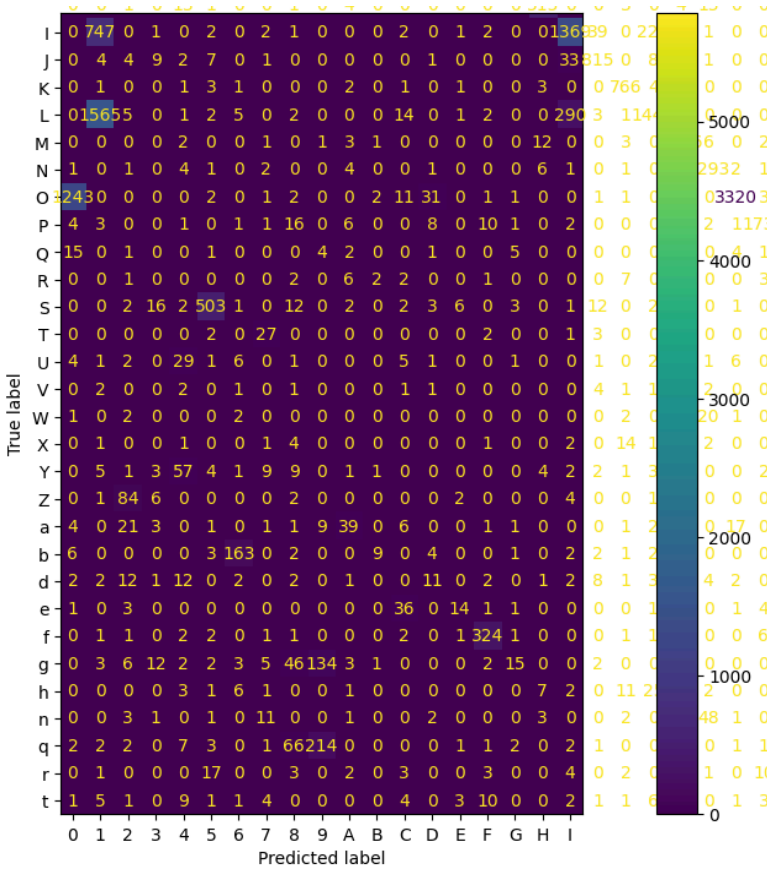
Esta matriz se puede interpretar de manera que, cuanto más claro este la celda (valor cercano a 1), quiere decir que el valor del eje X se confunde con el valor del eje Y, por ejemplo, si vemos el número 1 en el eje X, podemos observar que se suele confundir a veces con la letra “l”, y tiene sentido, ya que visualmente se parecen

Otro ejemplo es el número 9, que se suele confundir con la letra “q” de manera más frecuente que el anterior

Otra matriz de confusión que he creado es, en lugar de colores, con números, pero como es obvio, no es tan legible debido a las dimensiones

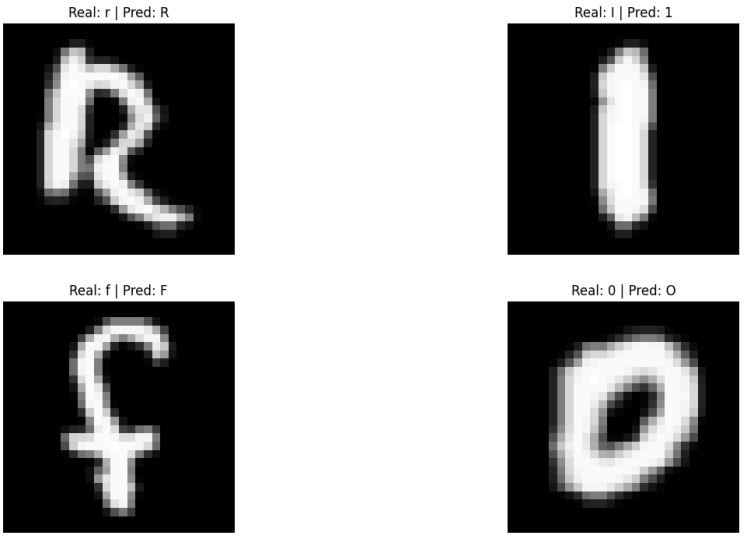


Pero si hacemos zoom en la parte inferior izquierda, si podemos interpretar lo que nos devuelve



Vemos que el número 1, se confunde exactamente unas 747 veces con la “I” en los datos de test. También podemos ver otros valores, como que el número 5 se suele confundir unas 503 veces con la letra “S”

Por último, vamos a ver dónde ha fallado el modelo y cuál era la respuesta correcta



Aquí vemos cuatro ejemplos, los cuales son fallos que puede ser normal, puesto que la “F” y la “f” quizás se puede confundir al tener una forma parecida, igual ocurre con el 0, donde ha predecido que es la letra “O”

Viendo estos cuatro fallos, podemos decir que el modelo es bueno, ya que los fallos no son muy graves, como por ejemplo confundir una “A” con un “0”, donde las formas son completamente diferentes, sino que tiene fallos con letras y números donde la forma son parecidas

TIEMPOS DE CONVERGENCIA

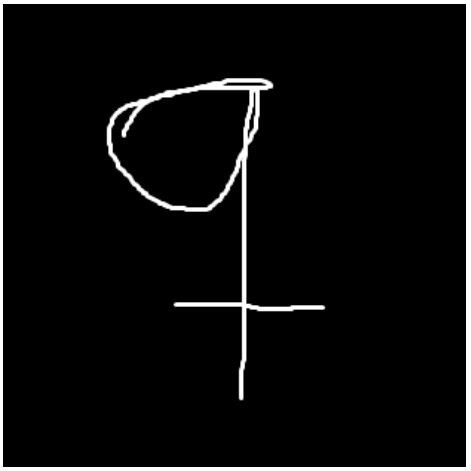
A continuación, vamos a comparar los optimizadores clásicos como el SGD, con dos adaptativos, como son el Adam y el RMSprop

Si medimos el tiempo en cuanto a segundos, el optimizador SGD suele tardar menos que los otros optimizadores. De las tablas anteriores donde aparecen los tres, en 9 ocasiones el SGD ha tardado menos que los demás, el optimizador Adam ha tardado menos que los demás en 6 ocasiones y el RMSprop en 3 ocasiones. Si lo medimos en número de iteraciones (epoch), el único que hace tres iteraciones cuando tiene la función de activación ReLU y la tasa de 0.001, es el optimizador Adam, los demás siempre hacen dos iteraciones. Hay que recordar que este apartado es solo un estudio del tiempo, ya que, aunque Adam tarde más, es el que suele obtener mejores resultados como ya hemos visto

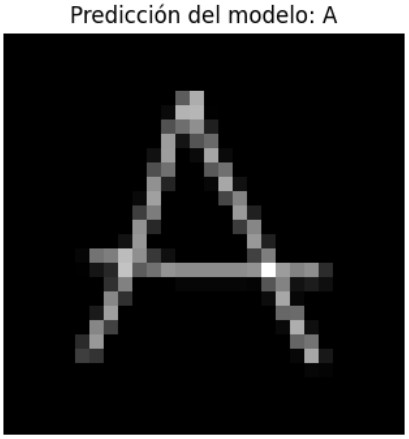
En conclusión, si solo nos fijamos en el tiempo, es mejor el SGD (optimizador clásico) con respecto a los demás, aunque el RMSprop también es una muy buena opción si queremos calidad/tiempo debido a que nunca supera las dos iteraciones y no suele pasar de los 65 segundos para obtener sus mejores resultados

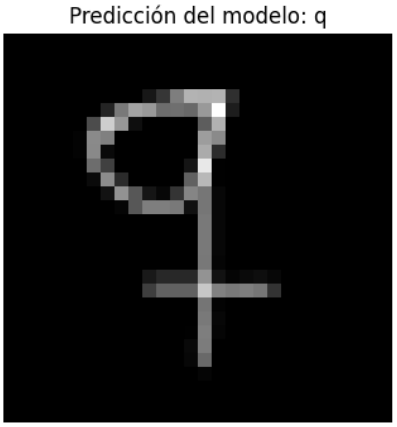
RECONOCIMIENTO EN TIEMPO REAL (SIMULADO)

Veamos ahora cómo se comporta el modelo con los parámetros óptimos con una imagen externa creada en Paint, las imágenes originales son las siguientes:



He elegido estas dos letras ya que quería comprobar cómo se comporta con una letra fácil de identificar (A), y con otra que se puede confundir con otros números, en este caso, la letra “q” puede confundirse con un 9 si no está bien entrenado. Es importante que se haga un preprocesamiento, en el que pase a una dimensión 28 x 28, con una escala de grises. Luego normalizamos los píxeles entre 0 y 1, y lo pasamos a un array de vectores de una dimensión de 784 elementos (28x28=784), de esta forma ya tienen el mismo formato que los del dataset, se lo pasamos al modelo y esto es lo que nos predice:





Vemos que ambas letras lo predice correctamente, incluso la letra “q” que puede llegar a confundirse con el 9

CONCLUSIONES

En conclusión, podemos afirmar que el modelo generaliza bastante bien y no tiene errores graves como hemos comentado anteriormente, donde si llega a fallar es en las letras que tienen formas similares. En cuanto al comportamiento de la red, podemos ver que en el entrenamiento, en cada iteración que hace el modelo (epoch), internamente hace un pequeño ajuste de los parámetros internos, esto hace que en la siguiente iteración empiece con una mejora en el accuracy

Por otro lado, en cuanto a las dificultades que se han superado haciendo la red han sido varias. Una de ellas es que para hacer la búsqueda de parámetros, una forma era con KerasClassifier, que es compatible con GridSearchCV, el problema es que, como se ha comentado al principio, KerasClassifier esta obsoleto, por lo que he visto factible hacer la búsqueda manualmente mediante bucles, de esta forma podía controlar cuánto tardaba cada modelo en ejecutarse, para luego hacer el estudio de la convergencia

Otra dificultad ha sido el tiempo en el que he tardado en buscar los parámetros óptimos, en el que he tardado un total de 9 horas (1.5 horas por cada búsqueda ejecutando el programa unas 6 veces debido a cosas que he ido añadiendo)

Por último, los hiperparámetros que mejor funcionan es el optimizador Adam con una tasa de aprendizaje de 0.001 y en algunas ocasiones funciona bastante bien el optimizador RMSprop con una tasa de 0.01